

Breaking Black Box - Evals

Craig West

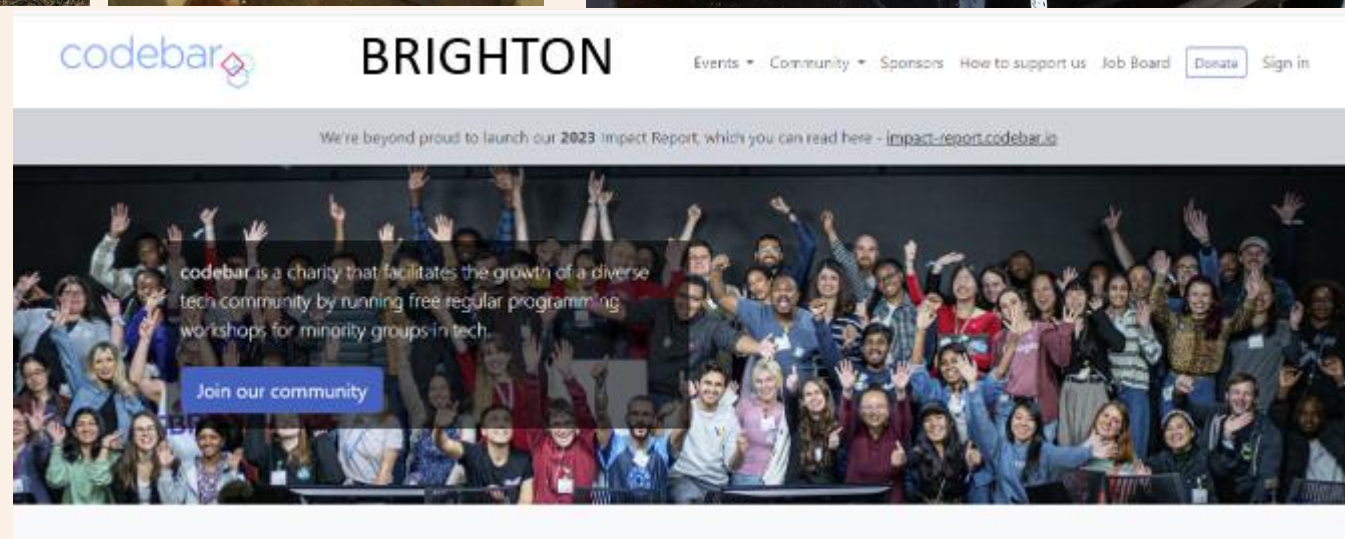
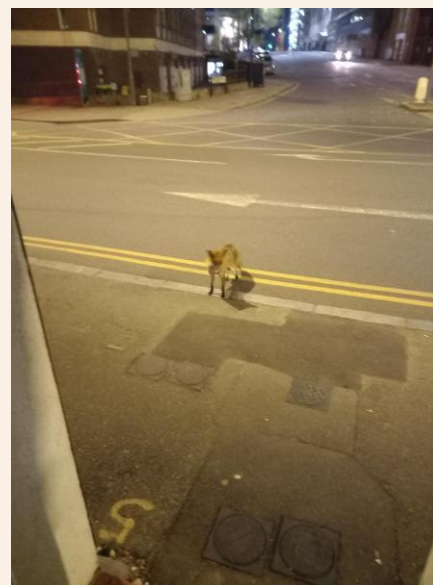
<https://craig-west.netlify.app/>

<https://evaluating-ai-agents.com/>

- Contains links for slides and eval-framework
- Everything in this talk is included



About me



My desired outcome

Not for you to fully understand Evals

But...

That Evals are doable

There are useful resources available for you

Demystify and simplify what an Agent is

Why Evals?

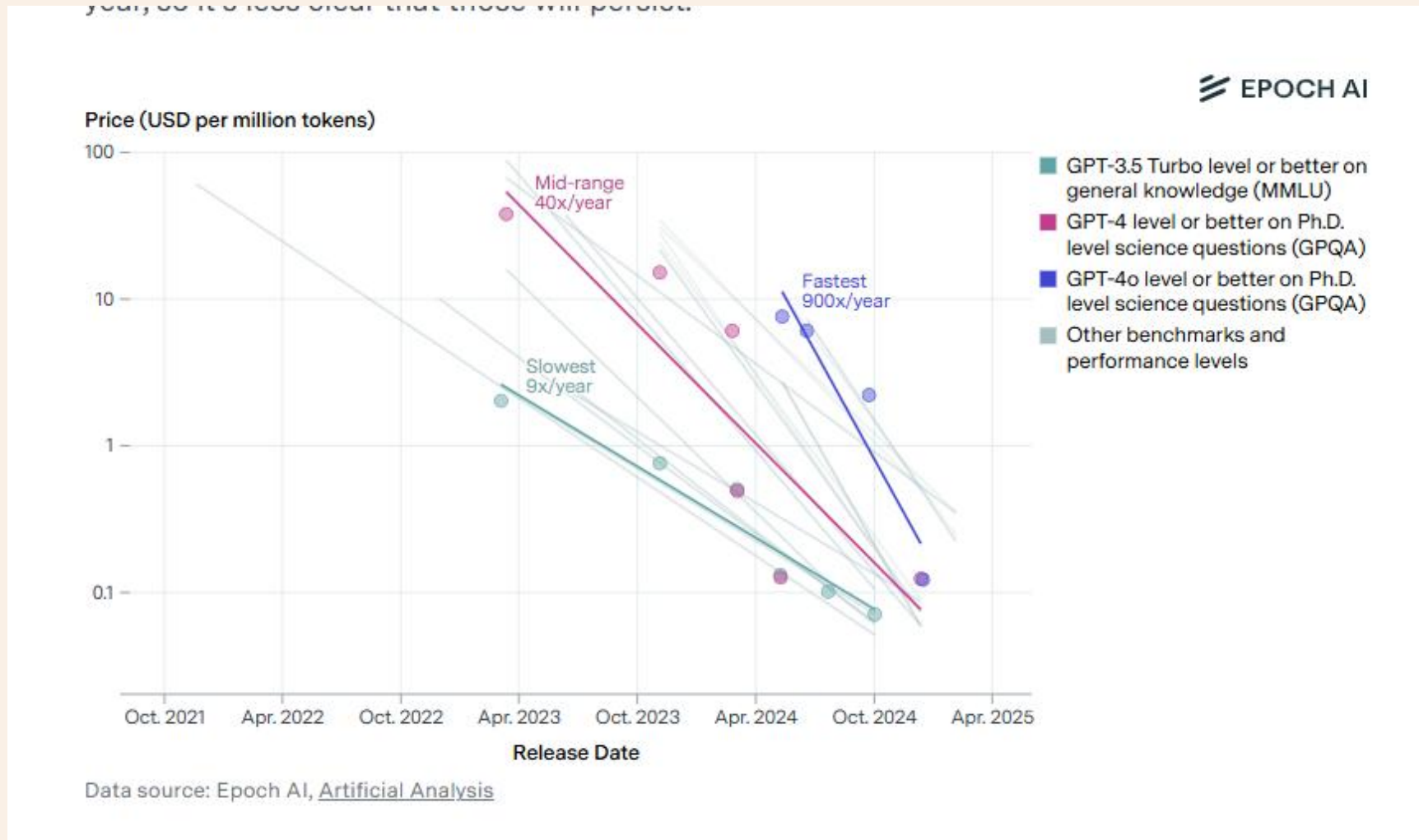
Ensure TRUST and CAPABILITIES of our AGENTS
so that somebody will pay for them.

Would you pay for a substandard product?

*Excellence in an evaluator will rub off on developer
prompt code.*

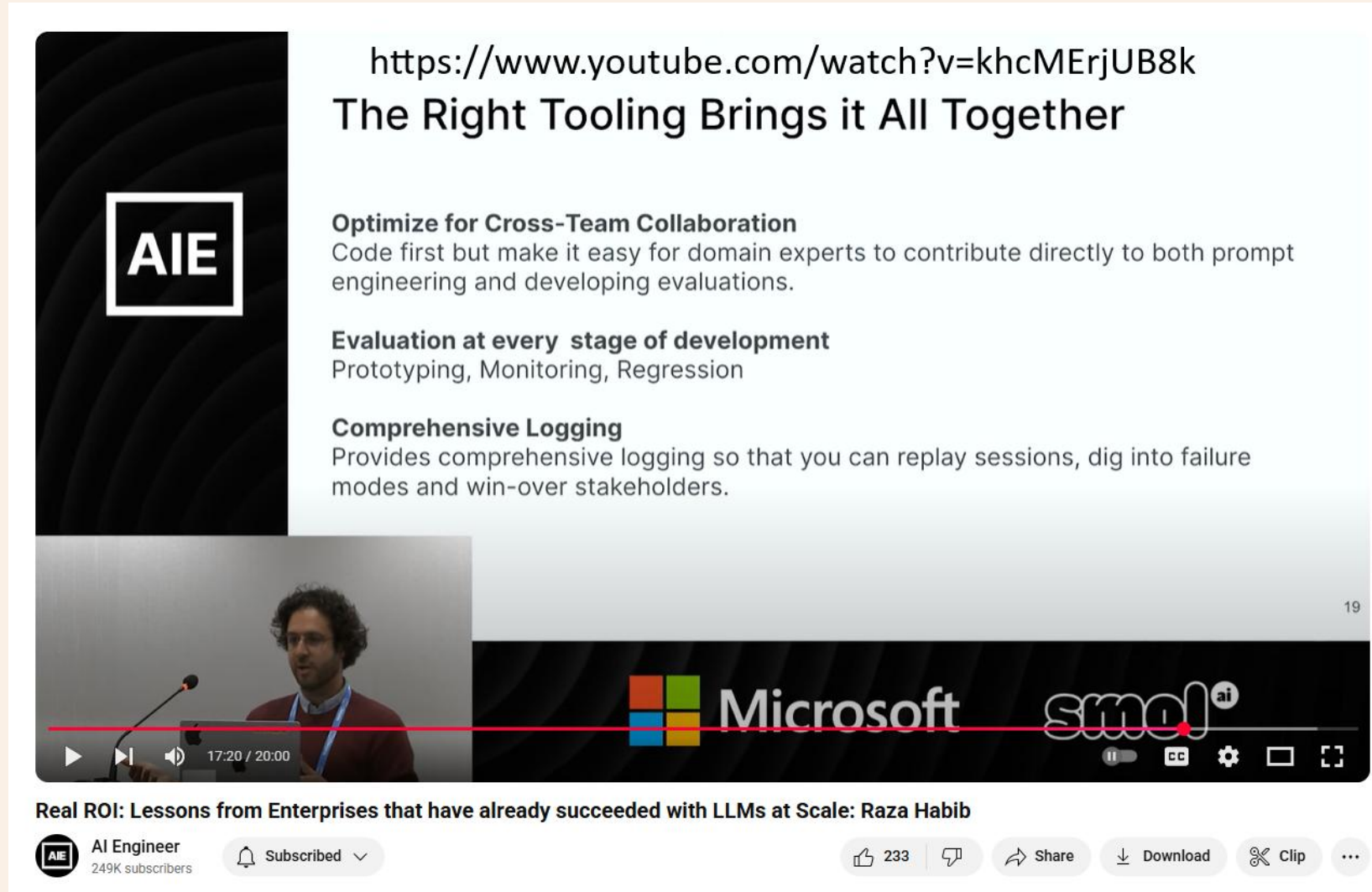
Can we save £ if we use a different LLM?

Model costs



Cost of a unit of 'intelligence has dropped ~ 99% in last two years and is 10x a year (anecdotal)

When to use Evals?



<https://www.youtube.com/watch?v=khcMErjUB8k>

The Right Tooling Brings it All Together

Optimize for Cross-Team Collaboration
Code first but make it easy for domain experts to contribute directly to both prompt engineering and developing evaluations.

Evaluation at every stage of development
Prototyping, Monitoring, Regression

Comprehensive Logging
Provides comprehensive logging so that you can replay sessions, dig into failure modes and win-over stakeholders.

19

Microsoft smol ai

17:20 / 20:00

Real ROI: Lessons from Enterprises that have already succeeded with LLMs at Scale: Raza Habib

AI Engineer
249K subscribers

Subscribed

233

Share

Download

Clip

Raza Habib is the CEO
and Cofounder of
Humanloop

Acquired by Anthropic

What is an AI Agent?

Many definitions and people opt for ‘Agentic Apps’

Anthropic

- **Workflows** are systems where LLMs and tools are orchestrated through predefined code paths.
- **Agents**, on the other hand, are systems where LLMs dynamically  direct their own processes and tool usage, maintaining control over how they accomplish tasks.

API

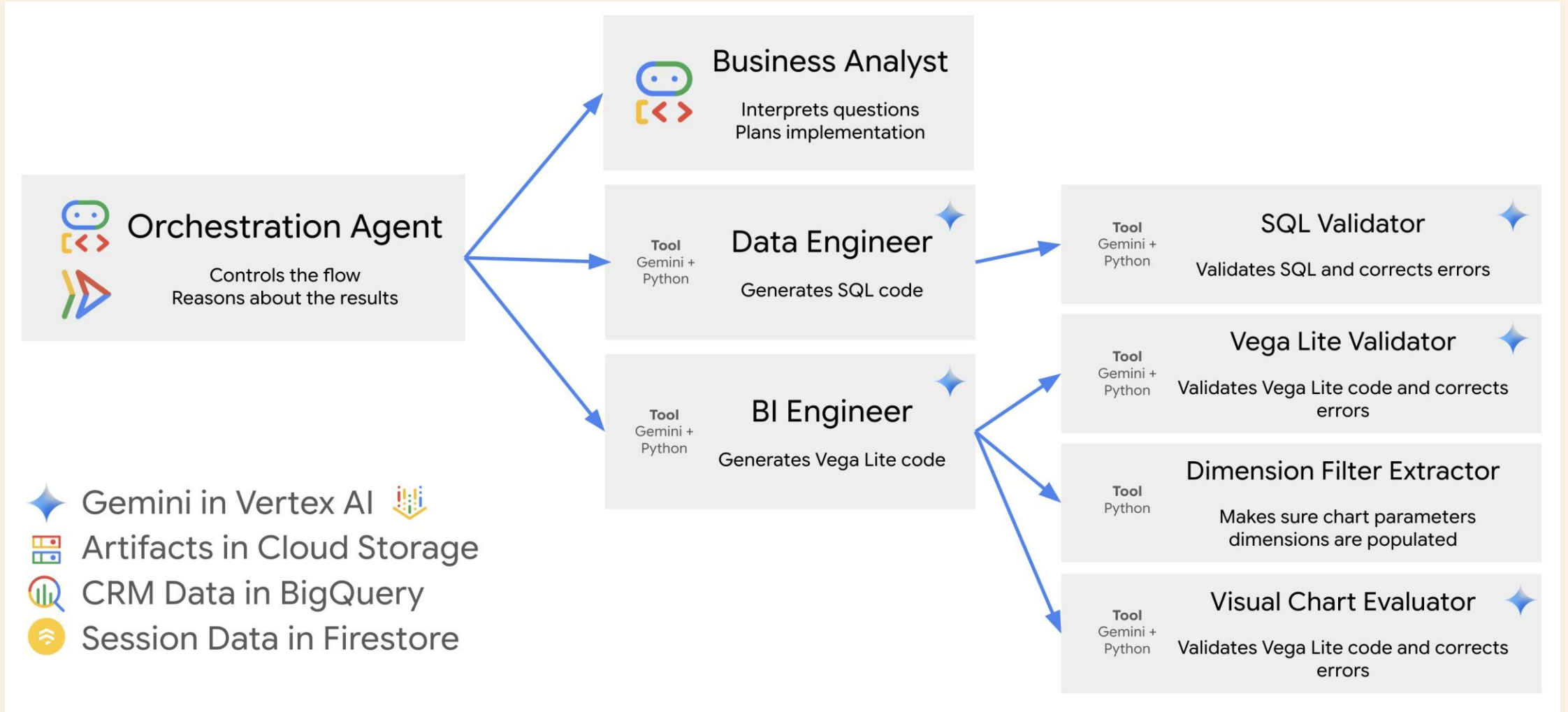
```
model = "gpt-4o-mini" # Model
model_endpoint = "https://api.openai.com/v1/chat/completions" # just one endpoint
headers = {
    "Content-Type": "application/json", # Authorisation
    "Authorization": f"Bearer {api_key}",
}
# payload structure may vary from LLM Organisation but it is a text string.
payload = {
    "model": model,
    "messages": [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    # additional parameters
    "stream": False,
    "temperature": temperature,
}
# Use HTTP POST method
response = requests.post(
    url=model_endpoint, # The endpoint we are sending the request to. Low Temperature:
    headers=headers, # Headers for authentication etc
    data=json.dumps(payload), # Inputs, Context, Instructions, Additional parameters
).json()
```

These messages contain
Instructions and Context

But I don't use Agents!

- Perhaps not now, but it might be that your app is required to use Natural Language, (text, voice) from the user.
- You might use or incorporate ready made 3rd party software that is Agentic as in the next slide...Google's CRM Agent...
- or use Model Context Protocol/Agent2Agent in your 'app'
- It may be helpful to understand what Agents are

Google CRM Agent



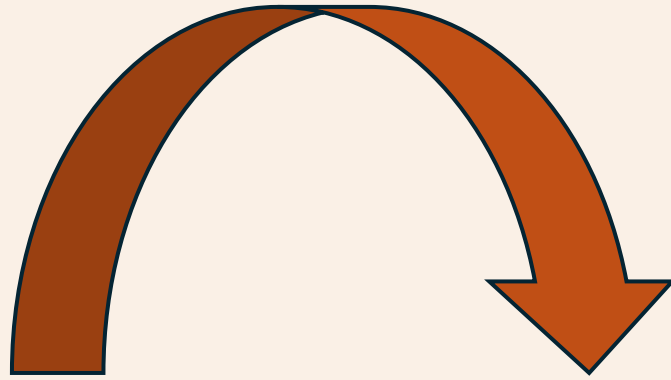
https://github.com/vladkol/crm-data-agent/blob/main/src/agents/data_agent/prompts/crm_business_analyst.py

<https://www.youtube.com/watch?v=2-AJszogj7Y>

What is Prompt/Context Engineering?

- Prompt Engineering is going out of fashion and people opt for *Context Engineering* and *In Context Learning*.
- We create a set of instructions and add **additional information** to a request we send to the LLM.
- We can let LLM know that more context is available if it tells us which of the **tools/functions** we have told it about it wants us to call with **arguments** it has provided – we will run it on our box and send the extra context/information.

180 degree shift



No more difficult than what we do currently – just 180 degrees different

“It doesn’t get any easier – just different” - Anon

Where to start? What to do?

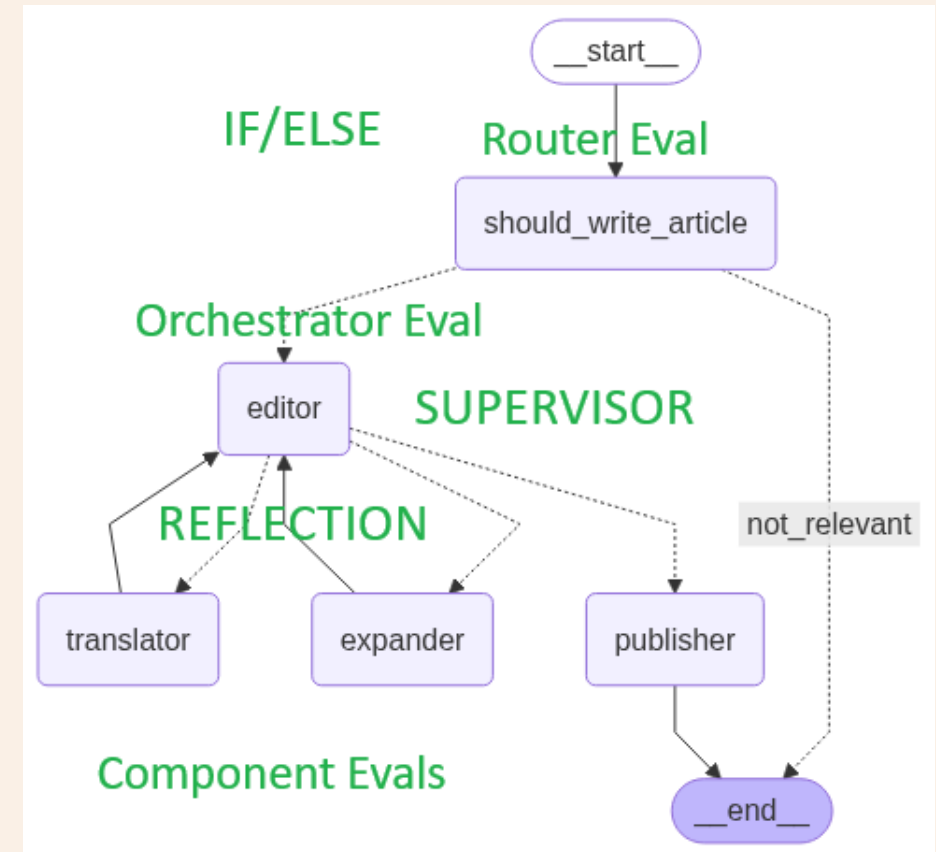
- We will look at Case Study 1 in the manual and framework.

Given an article title, determine if it is about AI

1. Take title and create article of N words (EXPANDER)
2. In a specified language (TRANSLATOR)
3. Ensure article is not sensational (EDITOR)

PUBLISH = YES only if all three are YES

Let's go to the guide and code...



A possible workflow

We build our app with a basic prompt “Write an article on {topic} of around {N} words.”

It all works mechanistically.

Now lets progress...

A possible workflow

We decide to add a simple 

We notice that users like:

Bullet points – Abstract - Conclusion

A possible workflow

We change our prompt to include these features.

However, only 2% of users give feedback so we decide to use Human Evaluators to give feedback as well as label articles 'good' etc

A possible workflow

We need to draft up guidelines on what an article should contain, its structure and other ways an article is 'good'.

This exercise further helps our LLM prompt and we refine it.

A possible workflow

We need to draft up guidelines on what an article should contain, its structure and other ways an article is 'good'.

This exercise further helps our prompt and we refine it.

A possible workflow

We can only handle/afford a small percentage, so we decide to use LLM as Judge.

With our original prompt and our human evaluator guidelines we write a prompt for this LLM Judge. We can now process a much greater percentage online and offline.

It is better than our app! We refine our app prompt...

A possible workflow

Now that we are looking at our data, we see that sometimes the app offers financial advice.

We must prevent this (guardrails) so we create an LLM as Judge that detects if this article had financial advice.

It saves many problems...

A possible workflow

Now that we have a robust way of evaluating our app, we can start to tinker:

- Different models in different places for cost reduction yet same performance?
- Does one part of the app give greater benefit for unit of change than another?
- Can we reduce latency without breaking anything (regression testing)
- Offer a client dashboard of filtered metrics to our clients in real time?

Demo – our app from earlier

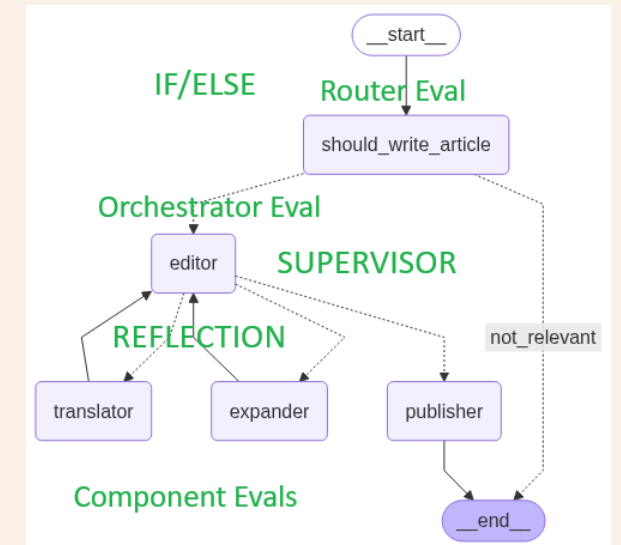
We will look at Case Study 1 in the manual and framework.

- Given an article title, determine if it is about AI (ROUTER)
- Take title and create article of N words (EXPANDER)
- In a specified language (TRANSLATOR)
- Ensure article is not sensational (EDITOR)

A ‘good article’ – OMITTED IN DEMO

PUBLISH = YES only if all three are YES in EDITOR

Let's go to the guide and code...



Summary Key Points

- We need to look at the data to determine what are EVAL criteria are
- We need to determine what success will look like
- We will need humans at certain points
- We need a Domain Expert
- LLM Judges are primarily for scaling
- We need to do EVALS on these LLM Judges
- Process more important than tools

Future

Craig West

<https://craig-west.netlify.app/>

<https://evaluating-ai-agents.com/>



Contact links on the sites

Always have 10 mins to answer questions...